

Bank of New York — Real-Time Merchant Settlement Intelligence Platform

Sprint Objective

The Data Engineers must take an **unseen banking problem** and build a working solution from **specification → data model → API → pipeline → front-end → testing → implementation**.

1. Business Scenario

Bank of New York provides payment acquiring services to merchants across India.

The Payments Operations team currently receives settlement information from multiple sources:

```
Payment Gateway
|
+----- Transaction System
|
+----- Settlement Processor
|
+----- Merchant Risk System
|
+----- Finance System
```

Operations currently uses Excel reports generated once every night.

The Head of Payments has identified a problem:

"Yesterday's transaction volume was ₹48 Cr, but only ₹45.8 Cr was settled. Operations cannot determine whether the difference is caused by failed payments, pending settlements, processing delays, merchant risk controls or data-quality problems."

The bank wants an engineering solution that allows operations users to:

1. View payment and settlement performance.
 2. Identify settlement gaps.
 3. investigate delayed settlements.
 4. identify merchants with abnormal settlement behaviour.
 5. expose the analytics through an API.
 6. consume the API through a simple web dashboard.
 7. process new data incrementally.
 8. automatically test the implementation.
-

2. The Unseen Data Sources

The data engineers receives four CSV files.

transactions.csv

Column	Description
transaction_id	Payment transaction
merchant_id	Merchant
customer_id	Customer
transaction_ts	Transaction timestamp
amount	Transaction amount
currency	Transaction currency
status	SUCCESS / FAILED / REVERSED
payment_channel	POS / ONLINE / QR

Expected grain:

One row = one payment transaction attempt.

settlements.csv

Column	Description
settlement_id	Settlement identifier
transaction_id	Payment transaction
settlement_ts	Settlement timestamp
settlement_amount	Amount settled
settlement_status	SETTLED / PENDING / FAILED
settlement_batch	Settlement batch

Important:

One transaction can have more than one settlement record.

merchant.csv

Column	Description
merchant_id	Merchant

Column	Description
merchant_name	Merchant
merchant_category	Merchant category
country	Country
risk_level	LOW / MEDIUM / HIGH
effective_from	Risk effective date
effective_to	Risk expiry date

The risk level is historical.

`payment_events.csv`

Column	Description
event_id	Event ID
transaction_id	Transaction
event_type	CREATED / AUTHORIZED / SETTLED / FAILED
event_ts	Event occurrence time
ingestion_ts	Time received by platform
processing_ms	Processing time

Some events arrive late.

3. Hidden Data Problems

The data engineers is **not told the following explicitly**.

They need to discover them during implementation/testing.

Issue 1 — One-to-many settlement

Example:

```
Transaction T1001
|
+-- Settlement S5001 = ₹8,000
+-- Settlement S5002 = ₹2,000
```

A direct transaction → settlement join can produce incorrect transaction-level calculations.

Issue 2 — Late-arriving events

Example:

```
event_ts      10:02:15
ingestion_ts  10:08:41
```

The data engineers must distinguish:

business time from **processing/ingestion time**.

Issue 3 — Out-of-order events

Example:

```
AUTHORIZED    10:01
SETTLED       10:03
CREATED       10:00
```

The physical arrival order does not necessarily represent business event order.

Issue 4 — Merchant risk changes

Example:

M100

```
LOW           Jan-Mar
HIGH          Apr-Jun
MEDIUM        Jul onwards
```

A transaction in February must use the February risk classification.

Issue 5 — Data-quality problems

The dataset contains:

- missing merchant IDs
- duplicate event IDs

- negative settlement amounts
- transactions without settlement
- settlements without a matching transaction
- invalid currency values
- events arriving after the expected SLA

The data engineers must decide which are:

- rejectable records
- quarantine records
- warning conditions
- business exceptions

4. Business Requirements

The Head of Payments wants a dashboard containing:

KPI 1 — Transaction Volume

Total successful payment value.

KPI 2 — Settlement Rate

$$\frac{\text{Settled transaction amount}}{\text{Successful transaction amount}} \times 100$$

KPI 3 — Settlement Gap

$$\text{Successful transaction amount} - \text{Settled amount}$$

KPI 4 — Settlement SLA

Percentage of settlements completed within **30 minutes** of the successful transaction.

KPI 5 — Merchant Risk

Number of merchants with:

Settlement Rate < 95%

and

5. Engineering Requirement

The bank does **not** want a single Python script.

The solution must have these layers:



The implementation should support **incremental processing** rather than reprocessing every record unnecessarily.

6. Sprint Tasks

TASK 1 — Specification Package

Before writing implementation code, the data engineers must produce a mini specification.

1. Business Specification

Document:

- problem statement
- objective
- users

- KPIs
- scope
- assumptions
- business rules
- acceptance criteria

2. Technical Specification

Document:

- data sources
- fields
- processing
- storage
- API
- front-end
- validation
- monitoring
- security

3. Gherkin

Write at least **5 acceptance scenarios**.

Example:

Scenario: Identify an unsettled successful transaction

Given a successful payment exists
 And no successful settlement exists for the transaction
 When the settlement pipeline runs
 Then the transaction should be classified as UNSETTLED
 And it should appear in the settlement exception report

TASK 2 — Data Model + Data Pipeline

Build the physical model.

A reasonable target may contain:

```
DIM_DATE
DIM_MERCHANT
DIM_CUSTOMER

FACT_TRANSACTION
FACT_SETTLEMENT
FACT_PAYMENT_EVENT
```

But the data engineers must justify the model.

Mandatory

The data engineers must explicitly document:

Grain of every fact table

For example:

```
FACT_TRANSACTION
One row = one payment transaction attempt
```

```
FACT_SETTLEMENT
One row = one settlement event
```

```
FACT_PAYMENT_EVENT
One row = one payment lifecycle event
```

Implement

- DDL
- primary keys
- foreign keys
- constraints
- indexes/partitioning where appropriate
- staging tables
- Bronze/Silver/Gold processing

Pipeline must handle

```
Raw
↓
Validation
↓
Clean
↓
Business transformation
↓
Gold
```

Invalid records should not silently disappear.

TASK 3 — API + Front-End + Full Test Suite

Part A — API

Create:

```
GET /api/v1/settlement-summary
```

Parameters:

```
start_date
end_date
merchant_id (optional)
```

Example:

```
GET /api/v1/settlement-summary?start_date=2026-09-01&end_date=2026-09-07
```

Response:

```
{
  "transaction_count": 15240,
  "transaction_amount": 482500000,
  "settled_amount": 458700000,
  "settlement_rate": 95.07,
  "settlement_gap": 23800000,
  "sla_rate": 91.4
}
```

Use:

- FastAPI
- Pydantic
- OpenAPI

Part B — Merchant Exception API

Create:

```
GET /api/v1/merchant-exceptions
```

Return merchants where:

```
settlement_rate < 95%
OR
settlement_sla < 90%
```

Example:

```
{  
  "merchant_id": "M1008",  
  "merchant_name": "ABC Retail",  
  "settlement_rate": 88.4,  
  "sla_rate": 72.5,  
  "risk_level": "HIGH"  
}
```

Part C — Front-End

Create a simple HTML/JavaScript dashboard.

It must show:

KPI cards

Transactions
₹ Transaction Volume
Settlement Rate
Settlement Gap
SLA Rate

Chart 1

Daily Transaction Amount
vs
Daily Settlement Amount

Chart 2

Top 10 merchants with settlement gaps

Table

Merchant
Risk
Settlement Rate
SLA
Settlement Gap

The front-end must consume the API rather than directly reading CSV files.

TASK 4 — Testing, Security, Deployment & Engineering Review

The data engineers must demonstrate that the solution is not just functional but engineered properly.

A. TDD / Unit Testing

Implement tests for:

- successful transaction
 - failed transaction
 - duplicate event
 - missing merchant
 - unmatched settlement
 - negative settlement
 - late event
 - settlement calculation
-

B. Data Model Tests

Include tests for:

Grain

```
SELECT transaction_id, COUNT(*)
FROM fact_transaction
GROUP BY transaction_id
HAVING COUNT(*) > 1;
```

Expected:

```
0 rows
```

Referential integrity

Transactions should not reference nonexistent merchants.

Settlement reconciliation

The candidate should be able to identify:

```
Successful transactions
+
Settlements
+
Unsettled transactions
```

and reconcile the totals.

C. API Contract Tests

Test:

```
200 - valid request
400 - invalid date
404 - unknown merchant
422 - invalid parameter
```

Also test that:

```
settlement_rate >= 0
settlement_rate <= 100
```

D. Security

The data engineers must identify and prevent issues such as:

- SQL injection
- unrestricted API access
- secrets inside source code
- PII leakage
- excessive logging
- missing authorization

For example, this should **not** be accepted:

```
query = f"""
SELECT *
FROM transactions
WHERE merchant_id = '{merchant_id}'
"""
```

The data engineers should use parameterized queries.

E. Deployment

Provide a deployment design:

GitLab

```

    |
    v
CI Pipeline
    |
    +-- Tests
    +-- SAST
    +-- Dependency Scan
    +-- Docker Build
    |
    v
DEV
    |
    v
TEST
    |
    v
PROD
```

The candidate should identify:

- environment configuration
 - secrets
 - health check
 - smoke test
 - rollback mechanism
-